

SD

Regression testing for conversational AI agents

Why it matters, and how to do it

A practitioner's point of view

EvalBrew AI Regression Test Harness

QualBrew Test Artifact Generation

Siddhartha Dhamankar

Global HR IT Innovation Lead

2026

Personal point of view · Not for distribution

Contents

1	Why regression testing is different for conversational AI	3
2	What we are actually testing for	4
3	Approach and methodology	6
4	The tooling: EvalBrew and QualBrew	8
5	Best practices distilled from doing it	10
6	Closing thought	11

1. Why regression testing is different for conversational AI

Traditional software is deterministic: the same input yields the same output, so a regression test is a simple equality check. Conversational AI agents break that assumption. The technology is **fundamentally non-deterministic**. The same utterance can produce differently-worded responses across runs, and a change intended for one topic can silently degrade an unrelated one. This is the central reason regression testing for these agents cannot be an afterthought. It has to be engineered as a discipline in its own right.

What forces a regression cycle

Across the Expert Assist to HR Assist to People Assist journey, the things that forced a regression cycle were rarely new features. They were changes underneath the experience:



LLM and model upgrades

Moving the response engine across GPT versions, each of which can shift tone, accuracy, and routing behaviour even when nothing else changes. A pushed LLM upgrade alone was enough to require a significant, unplanned testing effort.



Architecture changes

The shift from a legacy orchestrator to a modern platform, and the move from a topic-hierarchy model toward an agentic loop, each of which re-opens every prior assumption about behaviour.



Content changes

Editing question-and-answer pairs or the metadata that drives targeted responses, which can ripple into same-category and adjacent-category answers.



Platform and API instability

A roughly 10% API failure rate on calls the agent makes to produce a response, where the orchestrator silently falls back to a generic search agent. Without regression coverage and logging, this kind of degradation hides in plain sight.

The core insight. In conversational AI, you are not testing whether code still returns the same value. You are testing whether a non-deterministic system still behaves acceptably, accurately, safely, and consistently, after something underneath it changed. That is a measurement problem, not a pass/fail assertion.

2. What we are actually testing for

"Does it still work?" decomposes into several distinct questions, each needing its own treatment.

2.1 Orchestration and routing accuracy

Topmost and non-negotiable. Orchestration accuracy, meaning does the right query reach the right agent. It needs to be measured and reported as a first-class metric in its own right, separate from response quality. A response can be perfect and still be wrong if it came from the wrong agent. Only a routing-specific measure catches that failure.

When production testing showed targeted questions being mis-routed to a generic search agent rather than the intended HR agent, the failure was in routing, not in content.

2.2 Response accuracy and groundedness

Does the agent produce the correct answer, grounded in the sanctioned knowledge base, without fabricating? For employee-facing agents the bar is deliberately high: aim for full accuracy on factual responses, tolerate incomplete-but-safe answers, but never tolerate confidently wrong ones.

2.3 Personalisation correctness

As agents mature from generic (L1) to targeted (L2) to personalised (L3) answers, regression scope expands with them. An L3 answer computed against an individual's transactional data has to be re-verified whenever the data plumbing or the model changes. A correct policy answer with the wrong personal numbers is a regression.

2.4 Consistency at scale and across languages

The same intent must yield an equivalent-quality answer across runs, and across every language and geography deployed in. Structured language testing must be part of the plan, with a defined baseline, rather than assuming a model that works in English works everywhere.

2.5 Safety, responsible AI, and adversarial behaviour

Regression coverage cannot stop at the happy path. System accuracy testing, bias testing with SMEs, red-team testing of high-risk scenarios, and diverse user testing are all required, precisely because a model change can reintroduce a bias or a hallucination that a prior version had suppressed.

Negative scenarios, prompt-injection attempts, and hallucination probes belong in the regression pack, not a separate one-off exercise.

2.6 Non-functional behaviour

Response time, throughput, and error rate under concurrency are part of the regression surface too. Synthetic load testing is for finding hotspots. It is not a promise of production behaviour, and results must be reported honestly (P90, not dressed up as an average).

3. Approach and methodology

The approach that emerged from this work is layered and tiered, matching the depth of testing to the nature of the change.

3.1 Tier the regression to the change

Not every change deserves a full regression, and treating them the same either wastes effort or misses risk.

Change type	Trigger	Regression scope
1 Content edit	Q and A pair edits	Targeted: same/similar categories + random sample
2 Metadata change	e.g. entity codes, location tags	All affected pairs + random cross-category sample
3 Platform upgrade	LLM version, API, runtime	Full regression across all categories + bias testing
4 Architecture	Orchestrator, topic model, agentic loop	Full regression + routing benchmark + re-baseline

Figure 1. Tiered regression. Matching depth of testing to nature of the change

3.2 Measure, not just compare

Because responses are non-deterministic, string equality is the wrong test. The evaluation approach built into the harness compares each agent response against an expected answer using **semantic (cosine) similarity** with a configurable pass threshold. A differently-worded but equivalent answer passes, while a drifted or degraded answer fails. This is what makes regression meaningful for generative output.

3.3 Establish a baseline, then detect drift

Every regression is a comparison against a known-good state. Capturing a baseline, covering orchestration accuracy, response quality, and performance, *before* a change, means "did this get worse?" is an evidenced answer, not an opinion. Without a baseline, teams argue about anecdotes. With a baseline they reason about deltas.

3.4 Test at the right interface: the DirectLine pattern

To test a conversational agent the way an employee actually experiences it, the harness runs through the **DirectLine API**, programmatically opening a conversation, sending an utterance, and capturing the returned activity. Two design choices make this work at enterprise scale:

- **Non-interactive authentication** using client-credential flow, so batches run unattended with no human login.
- **Identity-driven, batched execution.** Each utterance runs under a test ID (an enterprise user identity), which the agent uses to fetch user-specific context and produce a personalised response. This lets a single batch exercise many personas and validate personalisation, not just generic answers.

3.5 Separate content issues from technology issues

A recurring, hard-won lesson: when a regression fails, the first question is whose failure is it? Drawing a clear line between **content issues** (owned by business and content teams) and **technology issues** (owned by engineering), reinforced by robust logging, ensures failures are triaged to the right owner rather than defaulting to ambiguity. Regression testing without this separation just generates noise.

4. The tooling: EvalBrew and QualBrew

Rather than wait for an off-the-shelf answer, the tooling was designed and built from scratch. because the business needed something easy to use, end-to-end, and enterprise-ready, not a toolkit that still needed development before it could run a single regression.

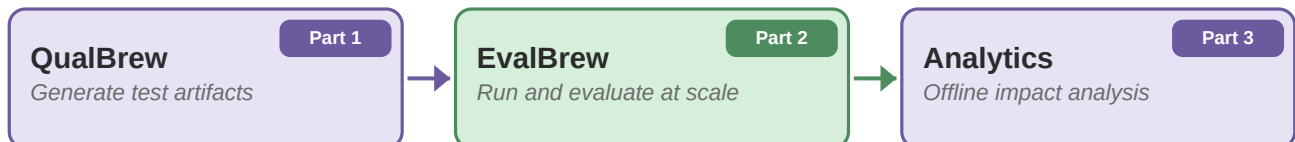


Figure 4. The two-part quality pipeline, with a third analytics component completing the ecosystem

4.1 EvalBrew. The AI regression test harness

EvalBrew is the regression engine. Its purpose is to execute a large predefined set of utterances (on the order of 2,000 to 6,000, each mapped to a test ID), systematically capture the agent's responses, evaluate them, and enable downstream analysis of impact after a feature, content, or platform change.

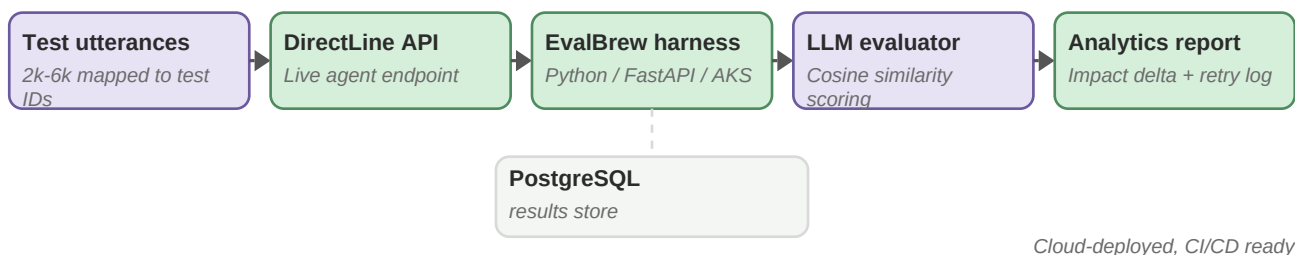


Figure 2. EvalBrew architecture. Test execution and evaluation pipeline



Built as a real service, not a script

Python and FastAPI, containerised, deployed to AKS with a PostgreSQL backing store and environment-driven configuration for CI deployments. Cloud deployment is treated as a prerequisite, because it removes the access and infrastructure barriers that otherwise kill adoption.

✓ Connects to live agents

Through streaming endpoints for response services and a dedicated LLM-evaluator endpoint. The harness scores responses against expected answers using cosine-similarity comparison with configurable thresholds, rather than brittle exact-match.

✓ Designed for extensibility

Positioned explicitly as an enterprise agent testing ecosystem, not test automation for a single use case. The extensible connector and evaluator framework is the whole point: a connector encapsulates a multi-step workflow behind a single interface, and custom evaluators can be added for use-case-specific assessment that no out-of-the-box tool provides.

4.2 QualBrew. Test artifact generation

Regression is only as good as the test cases feeding it. QualBrew is the front of the pipeline. It generates test artifacts (strategy, scenarios, cases, and data) from documented product knowledge, in plain natural language, so a business user with no testing background can produce a complete, grounded test set in minutes.

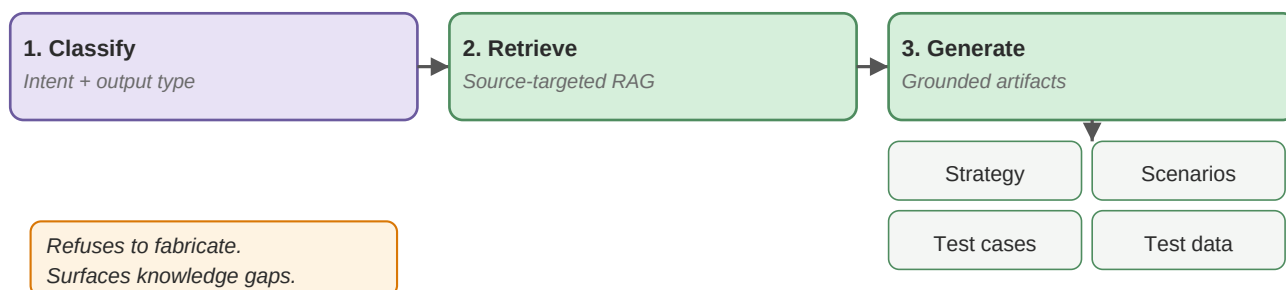


Figure 3. QualBrew agentic loop. Classify, retrieve, generate

✓ Classify, retrieve, generate

An agentic loop with an intent classifier that resolves the feature and output type and asks at most one targeted clarifying question, never more than two, before proceeding on a best-effort basis.

**Grounded, not fabricated**

Generated artifacts must reference actual documented acceptance criteria. The platform refuses to invent product specifics and instead reports the knowledge gap. The same anti-hallucination discipline demanded of the agents themselves, applied to the test generator.

**Output-type-targeted retrieval**

Strategy pulls from release notes, scenarios from user stories, cases from acceptance criteria, and data from a filtered subset. Each artifact type is grounded in the right source material, not a generic knowledge dump.

**Engineered agentic foundations**

Session memory, a vector store, a product CMS, and disciplined context-window slot management, with fixed token budgets for system prompt, running summary, retrieval package, and current message. This keeps long multi-turn sessions within model limits without losing context. These are the patterns that separate a demo from a solution that actually works.

5. Best practices distilled from doing it

The following principles are what I would give anyone standing up regression testing for a conversational AI agent, learned the hard way on live global deployments.

1	Treat regression as measurement, not assertion. Non-deterministic output means semantic, threshold-based evaluation. Never exact-match. Always compare against a captured baseline.
2	Make orchestration accuracy a first-class, separately reported metric. For any multi-agent system, routing is the failure mode that hides behind good-looking answers. Measure it on its own.
3	Tier the regression to the change. Targeted regression for content and metadata edits. Full regression with bias testing for LLM and platform upgrades. Re-baseline for architecture changes. Do not over-test the trivial or under-test the global.
4	Test through the real interface, unattended, at scale. Use the DirectLine or equivalent API with non-interactive auth and identity-driven test IDs so batches exercise real personalisation across thousands of utterances without a human in the loop.
5	Cover the unhappy path deliberately. Negative scenarios, prompt injection, hallucination probes, bias, and red-teaming belong inside the regression pack. A model change can reintroduce any of them.
6	Do not confuse synthetic performance results with production reality. Load tests find hotspots. Report them honestly with P90 and stated assumptions, and never let them masquerade as guaranteed production experience.
7	Separate content failures from technology failures. Log enough to tell them apart. Regression output is only actionable if each failure routes to the right owner. Invest in logging and observability up front. Gaps there make every regression ambiguous.
8	Build the tooling as enterprise infrastructure, not a point solution. Cloud-deployed, easy to use, end-to-end, and extensible through connectors and evaluators. A tool that needs a project before it can run a test will never be adopted by the business.

9	Ground your test generation the same way you ground your agent. Generate test cases from documented acceptance criteria, refuse to fabricate, and surface knowledge gaps. Otherwise you are testing the agent against fiction.
10	Regression is continuous, because change is continuous. With sprint-cadence deployments and a steady stream of content, model, and platform changes, regression capability has to be a standing, reusable asset. Run it every wave, not a one-time gate.

6. Closing thought

Conversational AI moved HR from processing tickets to holding conversations. But a conversation that used to be right and is now subtly wrong is far more dangerous than a hard error, because it still looks like success.

Regression testing is what protects trust as these systems change underneath us. It is not a QA checkbox at the end. It is the measurement discipline that lets a non-deterministic system be deployed, upgraded, and scaled with confidence. That conviction is why EvalBrew and QualBrew were built. Quality had to keep pace with the speed at which these agents now evolve.

Quality is not a gate at the end of the pipeline. It is a standing capability that runs every wave, measures every change, and protects every deployment.

Drawn from direct work building and deploying the EvalBrew AI Regression Test Harness and QualBrew test generation platform, and from written positions on orchestration accuracy, non-determinism, tiered regression, and enterprise test tooling (2024-2026). Views expressed are personal.